COURSE PROJECT REPORT

Low-Cost IoT-Based Multi-Sensor Air Quality Monitoring with Real-Time Cloud Visualization and Local Alerting

Subject: Design of Smart Cities

Team members

Arijit Majumder (23BCT0021)

Diya Aggarwal (23BCT0035)

School of Computer Science and Engineering, Vellore Institute of Technology

Vellore, Tamil Nadu, India – 632014

Contents

S.No	Section	Description	Page No.
—	Abstract	Summary of the project, the sensor node built and the key results obtained.	4
1	Introduction	Background, problem statement, intended users, objectives, and scope of the air quality node.	4
2	Software Requirements Specification (SRS)	Functional and non-functional requirements, use cases and acceptance criteria.	5
3	Literature Review, Research Gaps and Novelty	Review of existing IoT air monitors, the gaps found, and what is new in this work.	6
4	Project Management Plan and Execution Record	Work breakdown, execution record (preheating, live run, alert test) and challenges.	7
5	Software Design Specification (SDS) and UML Models	Layered architecture, hardware design, five UML diagrams, software logic and AQI equations.	10
6	Results, Validation and Discussion	Measured vs expected values, cloud logs, alert validation, standard comparison and limitations.	18
7	Smart City Application Perspective	How the node could scale across a city: deployment, data use and citizen participation.	21
8	Conclusion and Future Work	Summary of outcomes and planned improvements.	22
10	References and Data / Source Links	Numbered reference list [1]–[19] and online sources used.	22
11	Conference Paper	Attached separately with the submission.	19
11	Plagiarism Report (attachment)	Attached separately with the submission.	

Project Links

S.No	Resource	Link
1	Source code (GitHub)	https://github.com/CodeWithDiya-18/IoT-Based-Pollution-Monitoring-System
2	Project documents (Google Drive)	https://drive.google.com/file/d/1fEKk4jc8aTCzhAKDzWFiRklnL0k9NJ-I/view?usp=sharing
3	ThingSpeak channel (live dashboard)	https://thingspeak.mathworks.com/channels/3284492/private_show
4	WHO – ambient air quality and health	https://www.who.int/news-room/fact-sheets/detail/ambient-(outdoor)-air-quality-and-health
5	Central Pollution Control Board (CPCB)	https://cpcb.nic.in
6	CPCB – National Air Quality Index	https://www.cpcb.nic.in/national-air-quality-index/
7	Arduino Uno board documentation	https://www.arduino.cc/en/Main/arduinoBoardUno

Abstract

The issue of poor air quality is one of the least tangible problems in a rapidly growing city, and the monitoring stations capable of exposing it are both expensive and rare. This project designs and evaluates a cost-effective air pollution monitoring node built on an Arduino Uno, a DHT11 temperature–humidity sensor and MQ5 and MQ135 gas sensors. Readings are sent over Wi-Fi (ESP8266) to the ThingSpeak platform, where they are shown almost in real time. A buzzer, an LED and a 16×2 LCD warn nearby people when a threshold is crossed. The prototype was tested under real network conditions. Temperature and AQI values were reported to be consistent with reference data within 1 °C and 3 index points, and the alert mechanism operated as expected.

Index Terms: Air quality monitoring, Arduino, Internet of Things, smart city, ThingSpeak, MQ135, wireless sensor node.

1. Introduction

1.1 Background

Cities pack people, vehicles and industry into a small space, and the resulting air is often worse than what the same population would breathe elsewhere. Carbon monoxide (CO), methane (CH_4), ammonia (NH_3) and fine particles are linked to asthma, bronchitis and cardiovascular disease. The World Health Organization (WHO) estimates that almost nine out of ten children are exposed to particulate levels above its guideline. Knowing what is in the air at street and hour level is therefore a necessity for a city to be regarded as smart.

The Internet of Things (IoT) makes that awareness affordable. Small sensing boards can be networked, left unattended and read from anywhere, while the cloud handles storage and presentation. Environmental sensing is one of the most natural IoT applications in smart cities, because the data is continuous, location-bound and useful to many parties at once.

1.2 Problem Statement

Harmful gases such as CO, NH_3 and CH_4 are hard to detect with tools that are either costly or bulky. Existing low-cost units often lack remote access or an immediate warning, so users learn about a hazard only after exposure. This project designs a portable, inexpensive node that (i) senses temperature, humidity and harmful gases continuously, (ii) makes the data available remotely in real time, and (iii) warns people on the spot when a pre-set limit is exceeded, while staying simple enough to replicate across many city locations.

1.3 Intended Users and Stakeholders

Stakeholder	Need
Citizens and households	Real-time awareness of the air they breathe, especially children, elderly people and those with respiratory conditions

Schools and workplaces	Continuous checks for gas build-up and poor ventilation in indoor spaces
Municipal / pollution control authorities	Dense, real-time data beyond a handful of fixed stations (CPCB is cited as the reference)
Urban planners and researchers	Granular, location-tagged data for interventions and trend studies

1.4 Objectives

- Measure temperature, humidity, and the responses of CO, NH_3, CH_4 and ethanol-related gases with one low-cost node.
- Deliver the data through a full chain from sensors to a cloud dashboard over Wi-Fi on a fixed 15-second cycle.
- Provide an on-site alert (buzzer and LED) that works regardless of network availability.
- Evaluate the prototype against reference values and assess how the node could scale across a city.

1.5 Scope and Exclusions

The prototype is a benchtop unit. Its AQI is estimated from gas channels only, because no calibrated particulate channel is included. The cloud dashboard is ThingSpeak. The city-wide deployment is a design proposal and was not tested at that scale. Regulatory-grade measurement is outside scope.

2. Software Requirements Specification (SRS)

2.1 Functional Requirements

"Must" marks a core requirement and "Should" a desirable enhancement.

ID	Priority	Requirement	Status in prototype
FR-01	Must	Read temperature and humidity from the DHT11 sensor	Implemented and tested
FR-02	Must	Read gas concentration from MQ5 and MQ135 through analog inputs A0 and A1	Implemented; values estimated from datasheet curves
FR-03	Must	Convert raw readings to voltage, resistance, concentration and AQI	Implemented (Eqs. 1–5)
FR-04	Must	Check every reading against thresholds and activate the buzzer and red LED before transmission	Implemented; tested with elevated gas
FR-05	Must	Upload four data fields to ThingSpeak via HTTP POST every 15 s	Implemented
FR-06	Must	Show current readings on a 16×2 LCD	Implemented
FR-07	Must	Keep local alerts working without network access	Implemented by design
FR-08	Should	Show the dashboard and history from any internet-connected device	Implemented via ThingSpeak
FR-09	Should	Add a particulate (PM) channel with calibration	Future work (dust sensor fitted but not used for AQI)

FR-10	Should	Provide a mobile application for citizens	Future work

2.2 Non-Functional Requirements

Area	Requirement / target
Accuracy	Temperature within 1 °C and AQI within 3 index points of reference data (as reported)
Timeliness	Fixed 15-second upload cycle, the free ThingSpeak limit of one update per channel
Reliability	Local alarm must never be delayed by cloud upload; upload continues while alarm sounds
Cost	Off-the-shelf components only, to keep the bill of materials low
Portability	Works on any mobile Wi-Fi hotspot, so no fixed network is needed
Maintainability	Layered design so the Wi-Fi link or any sensor can be replaced independently

2.3 Use Cases

The use case model is shown in Section 5.3 (Figure 6). The main use cases are summarized below.

Use case	Actor	Expected result
View live readings	Citizen, school staff	Current values on the LCD and the ThingSpeak dashboard
Receive local alert	Anyone in the room	Buzzer and red LED on when a threshold is crossed
Review history	Citizen, authority, researcher	Time-series charts of the four ThingSpeak fields
Preheat and calibrate sensors	Node maintainer	Stable MQ outputs after warm-up and datasheet-based calibration
Estimate AQI	System	Overall AQI as the largest sub-index

2.4 Acceptance Criteria

- A reading above a threshold activates the buzzer and LED before the packet is transmitted.
- Every 15 seconds, one packet with four fields reaches the ThingSpeak channel.
- Measured temperature, humidity and AQI are compared with expected values and the differences are reported (Section 6.1).
- The node keeps running and alarming when the Wi-Fi link is unavailable.

3. Literature Review, Research Gaps and Novelty

Several groups have built IoT air monitors that differ mainly in the processor, the sensor set and how data is delivered. Jayakumar showed that real-time monitoring with Arduino and MQ sensors is feasible, but without trend anticipation. Gupta et al. addressed smart city coverage with little immediate alerting. Singh et al. kept cost low on Arduino but found it awkward to extend. Desai et al. moved to a BeagleBone Black

for more processing headroom at higher cost and complexity. Phala et al. used a standards-based approach without a cloud dashboard. Pal et al. logged pollution continuously without an analytical layer. Anitha and Kumar combined particulate and gas sensors on a Raspberry Pi with ThingSpeak reporting. Kondeti et al. reported a real-time air quality and weather station for sustainable urban development.

Together these studies show that sensing and logging are well covered. The combination that is less common is a cheap node that also raises a local alarm, publishes to a cloud dashboard, and can be multiplied across a city. That combination is the focus of this work.

3.1 Research Gaps

- Scalability: reference stations and single-node prototypes cannot be expanded cheaply to cover a city.
- Calibration and accuracy: MQ-series sensors need preheating and frequent calibration, and drift when left uncalibrated.
- High cost and complexity: reference-grade stations and boards such as the BeagleBone Black limit wide deployment.
- Weak sensor–cloud integration: many systems stop short of combining several sensors with real-time cloud visualization.
- No low-cost real-time alerting: most setups log data for later review and give no immediate on-site warning.

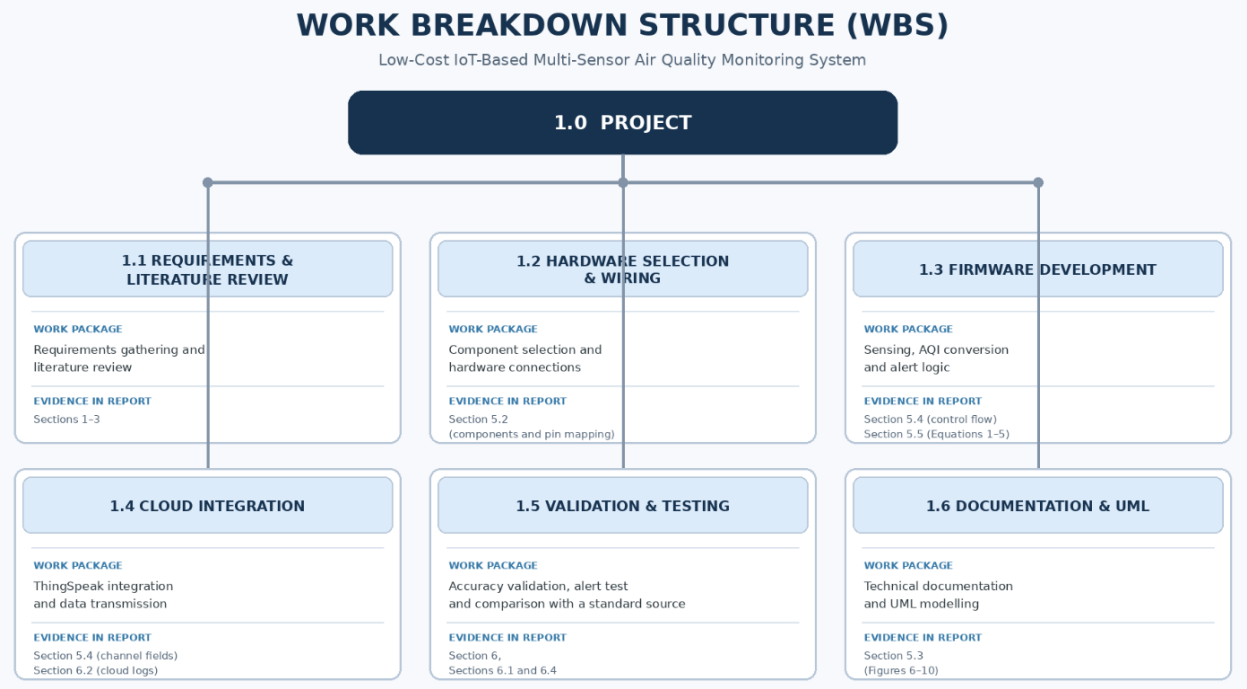
3.2 Novelty

The work addresses these gaps together in one node. Four design choices carry it: (1) fusion of DHT11, MQ5 and MQ135 on one Arduino board; (2) a cloud-first design in which an ESP8266 sends every reading to ThingSpeak; (3) real-time local alerts that trigger as soon as a threshold is reached; and (4) a low-cost bill of materials. Calibration drift is not eliminated, but it is managed through preheating, datasheet-based calibration and the validation in Section 6.

4. Project Management Plan and Execution Record

4.1 Work Breakdown

Phase	Work package	Evidence in this report
1	Requirements and literature review	Sections 1–3
2	Hardware selection and wiring	Section 5.2 (components and pin mapping)
3	Firmware: sensing, AQI conversion, alert logic	Section 5.4 (control flow), Section 5.5 (Eqs. 1–5)
4	Cloud integration with ThingSpeak	Section 5.4 (channel fields), Section 6.2 (cloud logs)
5	Validation: accuracy, alert test, comparison with a standard source	Section 6, Sections 6.1 and 6.4
6	Documentation and UML modelling	Section 5.3 (Figures 6–10)



4.2 Execution Record

- Preheating: all MQ devices were powered for 24 hours before the first measurement, as recommended by the manufacturers.
- Live run: after initialisation, the node joined a mobile hotspot and uploaded data with no manual intervention.
- Alert test: gas concentration was raised deliberately; the buzzer rang, the LED turned on, and uploads continued during the alarm.
- Logged window: a roughly 15-minute window was recorded on ThingSpeak (Figures 6.1–6.4).

4.3 Challenges and Resolution

Challenge	Response	Status
MQ sensors need long warm-up and drift	24 h preheat; datasheet calibration	Managed; long-term drift not studied
MQ sensors are not gas-selective	Values reported as estimates; AQI labelled informative, not regulatory	Stated limitation
DHT11 resolves only whole degrees	Accepted; ±5 % RH tolerance noted	Stated limitation
Calibration uses datasheet constants only	Not checked in a gas chamber	Open; future work
Dust sensor not calibrated against a reference	Particulate excluded from AQI	Open; future work
Cloud upload limit of one update per 15 s	15 s used as the sampling period	Resolved by design

4.4 Scheduling Diagram (Plan of Work)

The following Gantt chart is a proposed twelve-week relative schedule for the air quality node, not a record of exact dates. Tasks overlap intentionally: firmware development starts while the MQ sensors are still preheating, and the UML models are drawn while cloud integration and testing are under way.

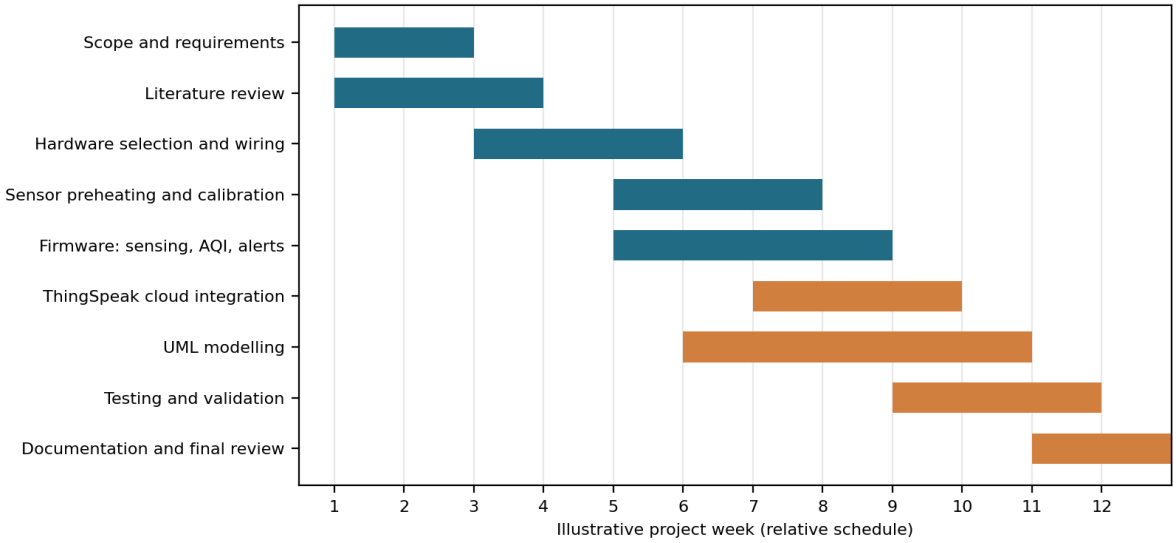


Figure 4.1. Illustrative Gantt chart of the project plan (relative weeks).

Milestones and acceptance gates

Milestone	Target	Completion evidence
M1: Scope approved	End W2	Objectives, requirements and scope (Sections 1–2); literature gaps identified (Section 3)
M2: Hardware assembled	End W5	Components and pin mapping (Table I, Section 5.2); wired prototype photographed
M3: Sensors calibrated	End W7	24-hour MQ preheat completed; manufacturer-based calibration coefficients and alert thresholds set
M4: Cloud link working	End W9	Four-field upload to ThingSpeak every 15 s; local alert logic running before transmission
M5: Validation complete	End W11	Alert test, logged ThingSpeak window and comparison with reference values (Section 6)
M6: Final submission	End W12	Report, UML diagrams, source code, results and limitations

Dependencies and critical path

Hardware wiring must precede calibration, because the MQ sensors need a 24-hour preheat before their readings can be trusted. Firmware development can proceed in parallel with preheating once the pin mapping is fixed. Cloud integration depends on working firmware and a Wi-Fi hotspot, and is limited to one update per 15 seconds by the free ThingSpeak tier. The likely critical path is hardware wiring → 24-

hour preheat and calibration → threshold and alert logic → ThingSpeak upload → validation → final report. A delay in preheating or in network access therefore affects every later task, so the sensors should be powered early.

Weekly control process

Hold one weekly review to compare planned and actual progress, list blockers and assign next actions. Every two weeks, demonstrate a tangible artifact such as a wiring photograph, a serial-monitor log or a ThingSpeak plot. Scope changes, for example using the fitted dust sensor in the AQI, should be assessed for their effect on calibration, schedule and documentation before approval.

4.5 Schematic Project-Management Workflow



Figure 4.2. Schematic project-management workflow. Each phase has a review gate; failed checks return to the previous phase.

The process begins by agreeing on the target pollutants, sensor set, alert thresholds and evaluation criteria. Hardware is then selected and wired, and each connection is checked against the pin mapping. After the 24-hour preheat, the MQ sensors are calibrated with manufacturer-based coefficients and the thresholds are set. The firmware and the ThingSpeak link are then integrated on a fixed 15-second cycle. Results are accepted only after the alert test and the comparison with reference values; a failed gate triggers corrective work rather than a premature claim of success.

5. Software Design Specification (SDS) and UML Models

5.1 Layered Architecture

The system has five layers, shown in Fig. 3 of the research paper. Data flows upward from the sensors to the user-facing application. Keeping the layers separate means that one layer, for example the Wi-Fi link, can be replaced without redesigning the rest.

Layer	Component	Responsibility
Sensing	DHT11, MQ5, MQ135	Produce temperature, humidity and gas signals
Processing	Arduino Uno (ATmega328P)	10-bit ADC, voltage and resistance calculation, threshold check, packet creation
Communication	ESP8266 Wi-Fi (802.11 b/g/n)	HTTP requests to the ThingSpeak API
Cloud	ThingSpeak	Storage, time-series charts, REST interface

Application	Web dashboard; LCD; buzzer; LED	Remote viewing and on-site alerts

5.2 Hardware Design

Component	Specification	Purpose
Arduino Uno	ATmega328P	Processing and control
MQ5 sensor	Analog output	Methane and combustible gases (analog A0)
MQ135 sensor	Analog output	CO, NH_3, ethanol, air quality (analog A1)
DHT11 sensor	Digital output	Temperature and humidity (digital D2)
ESP8266	Wi-Fi 802.11 b/g/n	Cloud data transfer (serial TX/RX)
16×2 LCD	I^2C (A4/A5)	Local display
Buzzer	5 V active	Audible alert (D9)
LED	5 mm	Visual alert (D8)

The ESP8266 communicates with the Arduino over serial lines. The buzzer and LED are driven from digital outputs D9 and D8. The hardware architecture with its pin mapping is shown in Figure 5.1, the block diagram in Figure 5.2, and the assembled circuit in Figure 5.3.

The MQ5 and MQ135 sensors connect to analog inputs A0 and A1 respectively, while the DHT11 sensor connects to digital pin D2. The ESP8266 module interfaces with the Arduino through the serial (TX/RX) pins, and the 16×2 LCD is connected over I2C (A4 = SDA, A5 = SCL) for real-time local output. The buzzer and the LED indicator are driven from D9 and D8. The 3.3 V regulator in Figure 5.2 supplies the Wi-Fi module, since the ESP8266 is not 5 V tolerant.

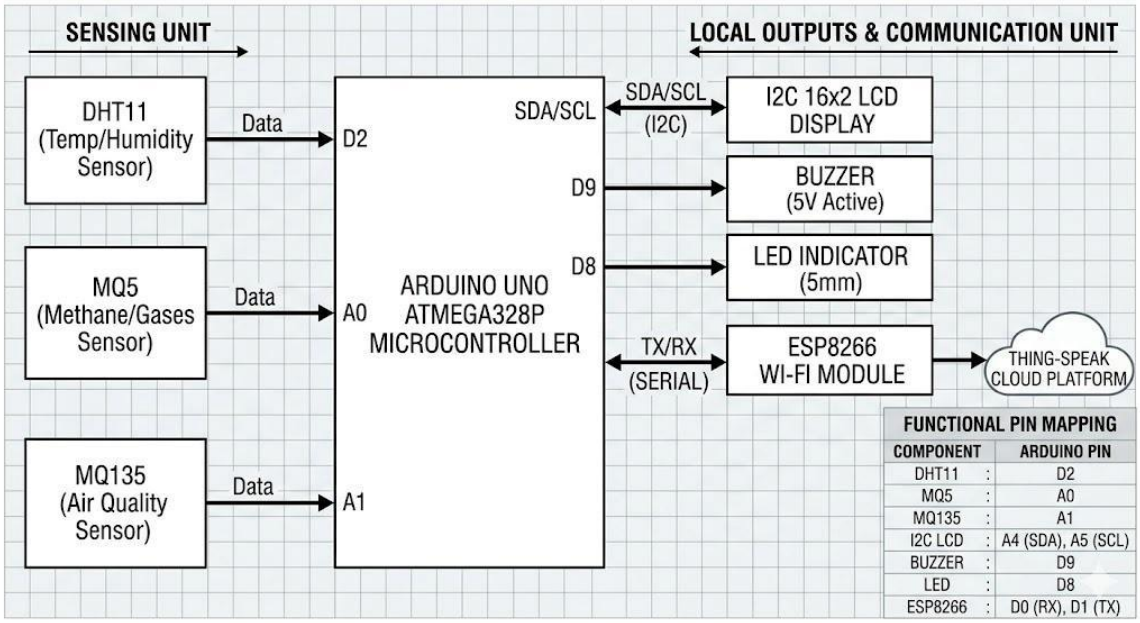


Figure 5.1. Hardware system architecture of the node with the functional pin mapping.

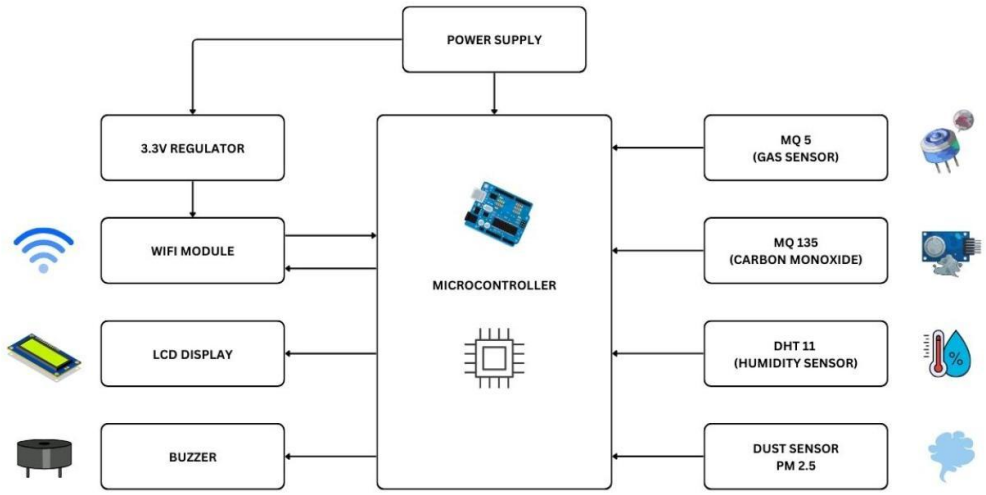


Figure 5.2. Block diagram of the monitoring node (power supply, 3.3 V regulator, Wi-Fi module, LCD, buzzer and sensors).

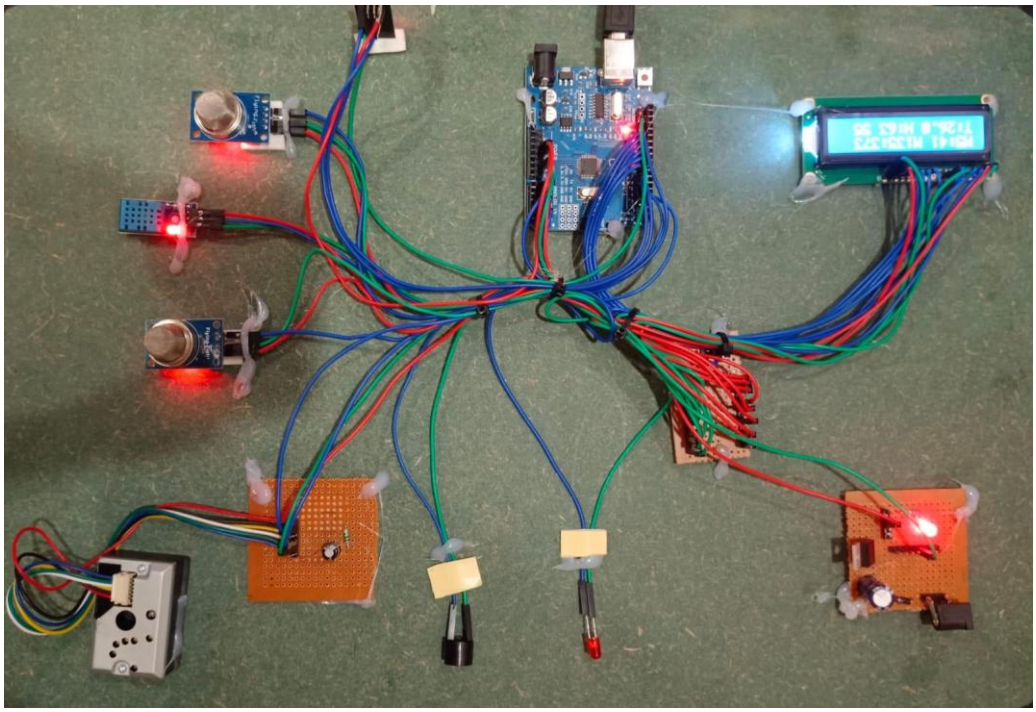


Figure 5.3. Circuit diagram: the assembled prototype with the Arduino Uno, DHT11, MQ5, MQ135, dust sensor, LCD, buzzer and LED.

The same wiring is used for the live tests reported in Section 6. In addition to the sensors listed in Table I, the prototype carries a dust sensor (PM2.5), visible in the lower-left corner of Figure 5.3; as stated in Section 6.5, its output is not calibrated and is not used in the reported AQI.

Sensor Calibration and Data Transmission Protocol

- MQ sensors require a preheating period of 24 hours for initial calibration.

- Calibration coefficients are determined from manufacturer specifications, and threshold values are set according to safe environmental limits defined by air quality standards.
- Data are sent over HTTP through the ThingSpeak API at an update interval of 15 seconds, which complies with the ThingSpeak free-tier limit.
- Data fields include temperature, humidity and the individual gas concentrations (parts per million).

5.3 UML Models

The five UML models below describe the node from different viewpoints: who uses it (use case), what it is made of (class), how a cycle flows (activity), how the node changes state (state), and how components exchange messages in one cycle (sequence).

5.3.1 Use Case Diagram

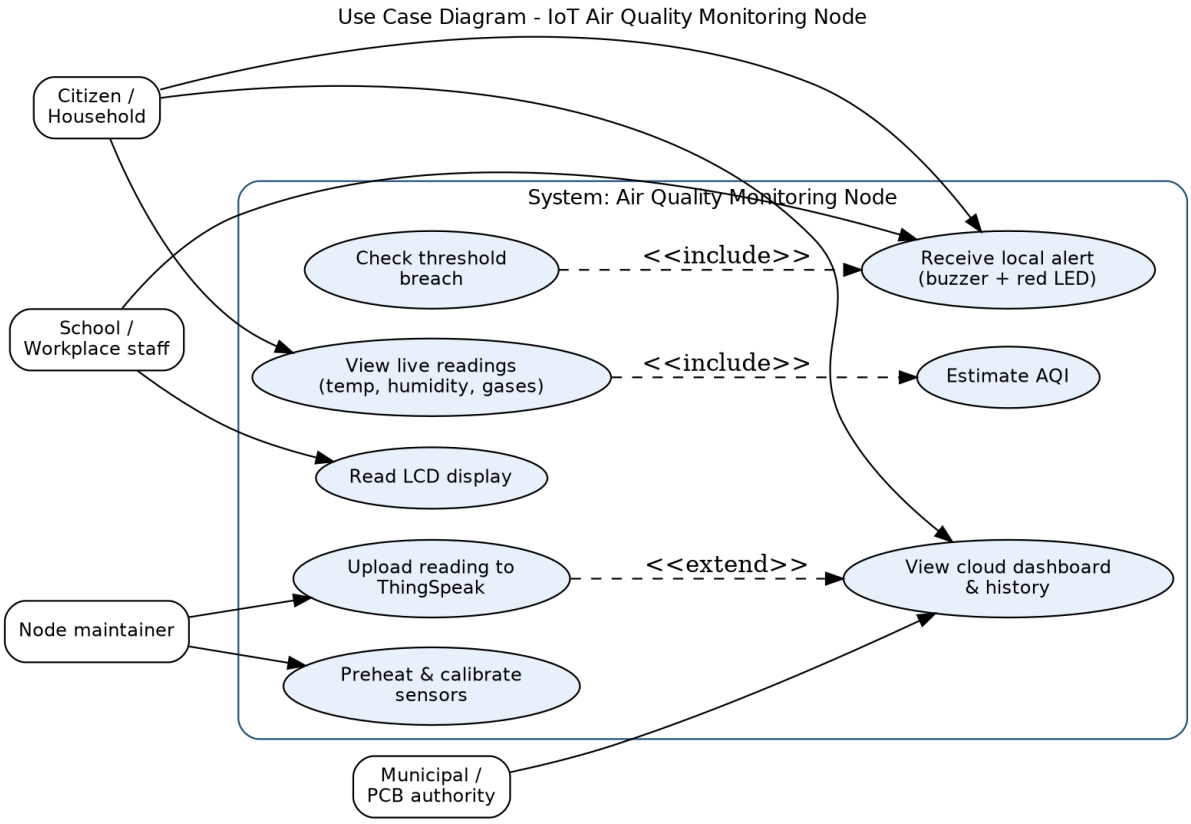


Figure 6. Use case diagram of the monitoring node.

The diagram shows four kinds of actor. A citizen or household member views live readings, receives local alerts and reviews the cloud dashboard. School and workplace staff read the LCD and respond to alerts. Municipal or pollution control authorities consume the dashboard data. The node maintainer preheats and calibrates the sensors and manages ThingSpeak uploads. Threshold checking is included in the alert use case, because an alert cannot occur without a check. The AQI estimate is included in live reading, and the dashboard upload extends the history view.

5.3.2 Class Diagram

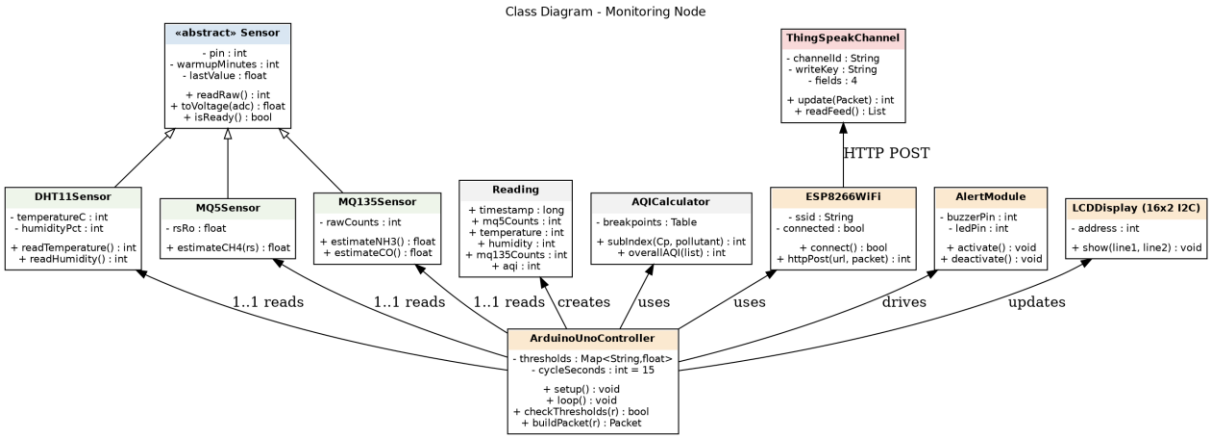


Figure 7. Class diagram of the monitoring node (attributes and operations shown).

Sensor is the abstract base for the three sensor classes, which share a pin, a warm-up period and a last-value attribute. ArduinoUnoController is the central class. It owns the sensor objects, creates a Reading for each cycle, uses AQICalculator to convert gas values, and drives the alert module, LCD and Wi-Fi module. ESP8266WiFi posts a packet to ThingSpeakChannel, whose four fields match the channel table in Section 5.4. The diagram follows the multiplicities that appear in the circuit: one controller and one of each sensor.

5.3.3 Activity Diagram

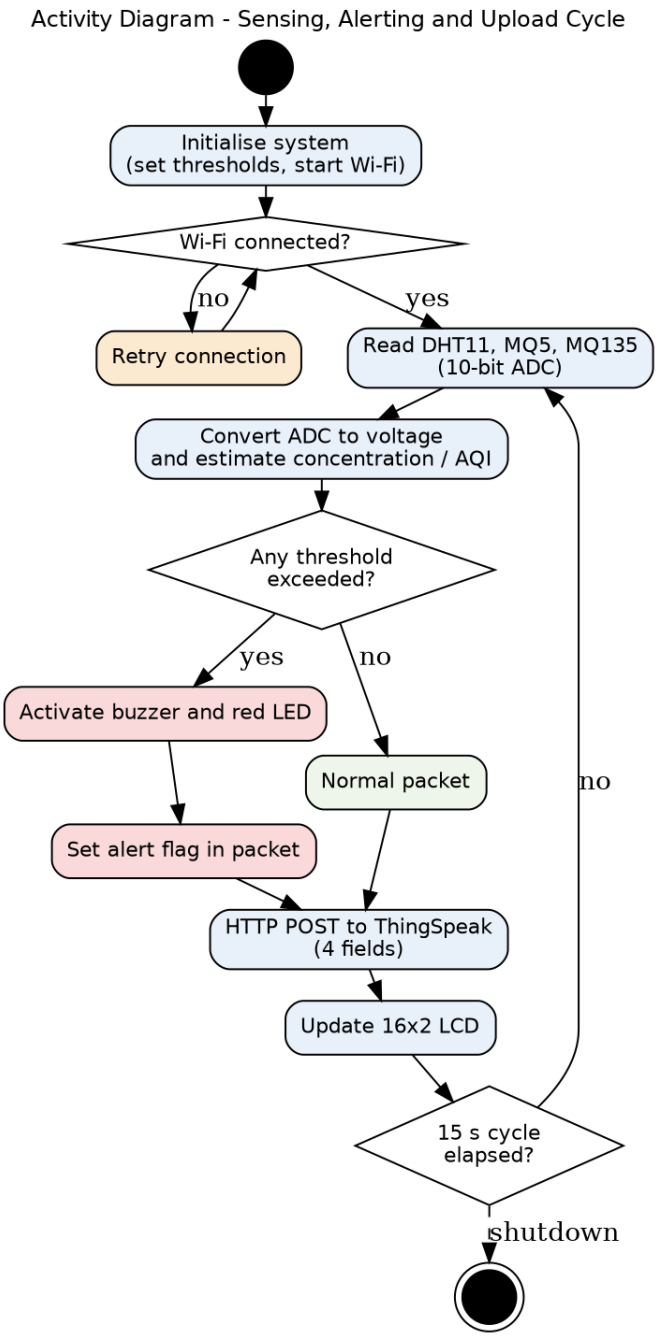


Figure 8. Activity diagram of the sensing, alerting and upload cycle.

The activity starts with system initialisation and waits for Wi-Fi. Each cycle reads the three sensors, converts the readings, and checks the thresholds. If any threshold is exceeded, the buzzer and red LED turn on and an alert flag is set before transmission. Otherwise a normal packet is prepared. Both paths transmit to ThingSpeak, update the LCD, and wait until the 15-second cycle has elapsed. This ordering reflects the design goal that local alarms never wait for the cloud.

5.3.4 State Diagram

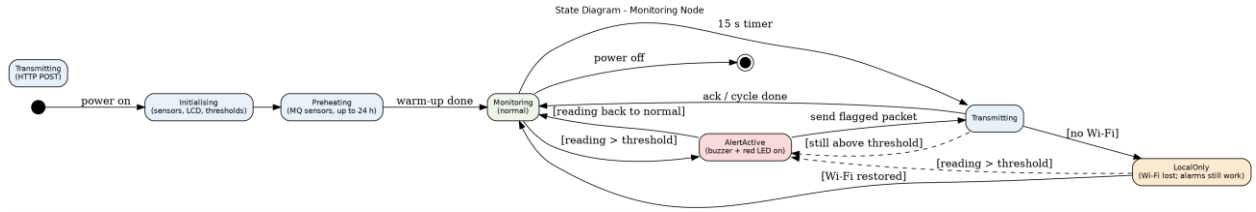


Figure 9. State diagram of the monitoring node.

The node moves from Initialising to Preheating, where the MQ sensors stabilise. It then enters Monitoring. A threshold breach moves it to AlertActive, where the buzzer and LED stay on until the reading returns to normal. Transmitting is entered on each 15-second timer, and the node returns to Monitoring afterwards. If the Wi-Fi link drops, the node enters LocalOnly. In that state the alarm still works, which reflects the requirement that local alerting does not depend on the network.

5.3.5 Sequence Diagram

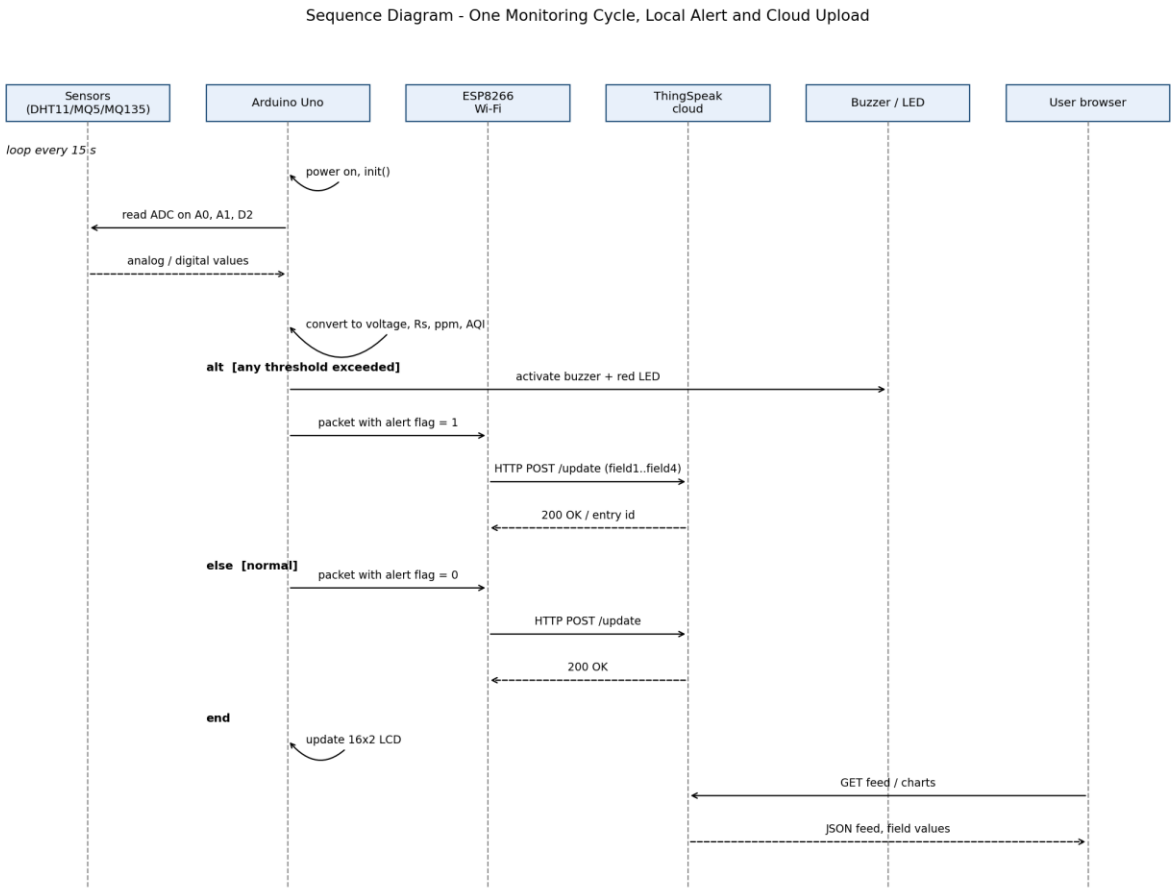


Figure 10. Sequence diagram of one monitoring cycle, from sensing to cloud and browser.

The sequence shows one cycle with its lifelines: the sensors, the Arduino, the ESP8266, ThingSpeak, the buzzer/LED and the user's browser. The Arduino reads the sensors and computes AQI. In the alarm branch, the buzzer is activated before the packet is sent, so the local warning does not wait for the upload. Both branches send a POST to ThingSpeak and receive an acknowledgement. The browser fetches the feed separately, which is why the dashboard can update independently of the node.

5.4 Software Logic and Alert Mechanism

After startup, the sensors are initialised and the thresholds are set. The device then enters an infinite loop of reading, formatting and testing values against thresholds. When any threshold is exceeded, the buzzer and red LED turn on before data transmission. In either case the data is sent over HTTP to ThingSpeak. Readings are uploaded with HTTP POST requests, and the free ThingSpeak tier accepts one update per 15 seconds per channel, which is therefore the sampling period. Four fields are configured, as listed below.

Field	Quantity	Unit
Field 1	MQ5 gas reading (analog)	ADC counts

Field 2	Temperature (DHT11)	°C
Field 3	Relative humidity (DHT11)	%
Field 4	MQ135 air-quality / ammonia reading	ppm (est.)

5.5 Calculation Logic

The 10-bit ADC maps the sensor voltage to an integer, and the voltage is recovered by Eq. (1). The sensor resistance is then found from the load resistor and supply voltage by Eq. (2). Gas concentration follows the log–log characteristic in the datasheet (Eq. 3). For a pollutant p, the sub-index is found by linear interpolation between the breakpoints (Eq. 4). The overall AQI is the largest sub-index (Eq. 5).

$$V = ADC \times 5 / 1023 \quad (1)$$

$$R_s = R_L (V_c - V) / V \quad (2)$$

$$\log_{10} C = m \cdot \log_{10}(R_s / R_0) + b \quad (3)$$

$$I_p = [(I_{HI} - I_{LO})/(BP_{HI} - BP_{LO})] (C_p - BP_{LO}) + I_{LO} \quad (4)$$

$$AQI = \max(I_1, I_2, \ldots, I_n) \quad (5)$$

Since the node has no calibrated particulate channel, the reported AQI is estimated from gas channels only and should be treated as informative rather than regulatory.

6. Results, Validation and Discussion

6.1 Measured versus Expected Values

Parameter	Expected	Measured	Unit	Remark
AQI	60	62	index	Very close
Temperature	35	34.8	°C	Accurate
Humidity	24	26	%	Slight variation
CO_2 (MQ135)	420–480	~450	ppm	Within range
CO	0.5–1.5	~1.0 (est.)	ppm	Safe level
NH_3	5–20	~10 (est.)	ppb	Very low
LPG / CH_4	1–5	~3 (est.)	ppm	No leakage
MQ135 raw	n/a	312	ADC	Air quality
MQ5 raw	n/a	148	ADC	Gas sensor

The temperature difference is 0.2 °C and the AQI difference is 2 index units. The humidity difference of 2 percentage points falls within the ±5 % RH tolerance of the DHT11. Values marked as estimated were derived from datasheet curves and are approximate.

6.2 Cloud Logs

On power-on the node initialises the sensor modules and Wi-Fi, joins the local network through a mobile hotspot, authenticates with the ThingSpeak platform and then begins continuous data acquisition and transmission. The ThingSpeak dashboard presents four plots: methane concentration (ppm) against time; carbon monoxide and ethanol concentration (ppm) against time; ammonia concentration (ppm) against time; and temperature (°C) and humidity (%) against time. The channel also supports geolocation tagging by latitude and longitude, so that readings can be shown against the location of the node. Figures 6.1 to 6.4 show the recorded results.

In the ThingSpeak logs, the MQ5 channel stays near 550 and then rises to about 600 before dropping at the end of the window. The temperature channel alternates between 24 and 25 °C, which corresponds to the one-degree resolution of the DHT11 crossing small real changes. The MQ135 channel fluctuates between 430 and 480 with a maximum near the end. Brief excursions on several channels at the same moment are consistent with a real disturbance near the node rather than independent sensor noise.



Figure 6.1. ThingSpeak field charts for the logged window: Field 1 (MQ5), Field 2 (temperature, °C), Field 3 (humidity, %) and Field 4 (MQ135).

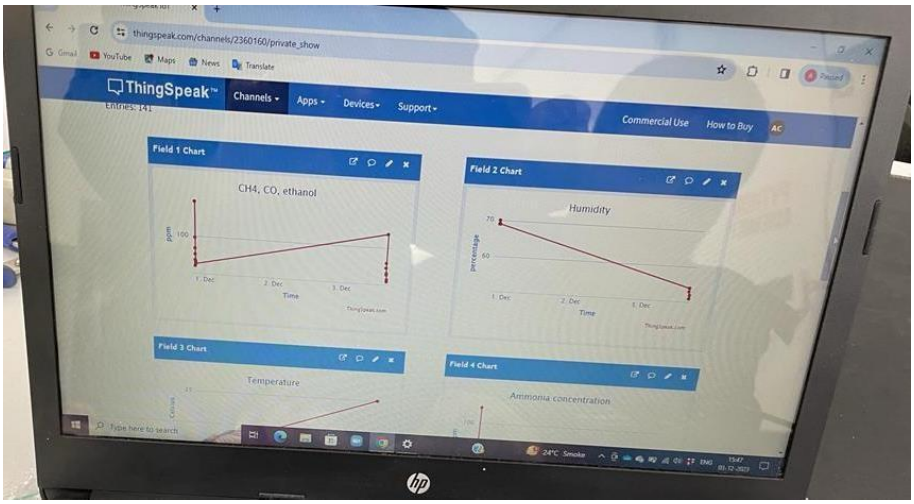


Figure 6.2. ThingSpeak dashboard showing the plots of methane, carbon monoxide and ethanol (ppm) and humidity (%) in the surrounding air.

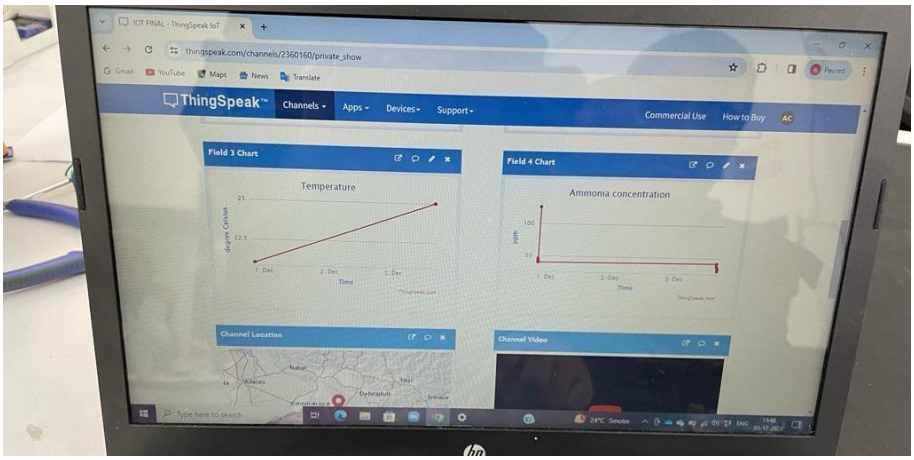


Figure 6.3. ThingSpeak dashboard showing the plots of temperature (°C) and ammonia concentration (ppm), with the channel location map.

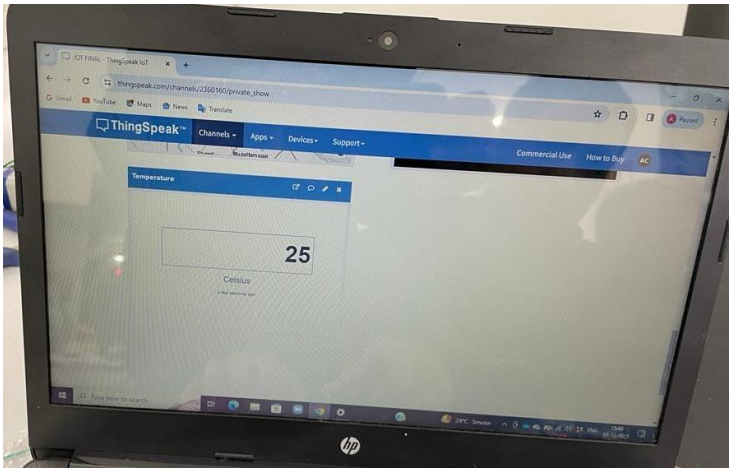


Figure 6.4. ThingSpeak widget showing the current room temperature in degrees Celsius.

The plots show how the environmental parameters vary over time, which lets a user identify trends and anomalies in air quality. Data acquisition was consistent, Wi-Fi connectivity was stable, data were visualised with minimal latency, and alerts were triggered whenever a threshold value was exceeded.

6.3 Alert Validation

To test the alert, the sensors were exposed to an increased gas concentration. Once the value passed the threshold, the buzzer rang and the LED turned on. Values continued uploading to the cloud while the alarm sounded, so alerts were available both locally and remotely.

6.4 Comparison with a Standard Source

The AQI was also tracked over several days against a value published by a standard source. The two series rise and fall together, and the node's value is usually within a few points of the published one. The gap is expected, because the node uses inexpensive MQ sensors whose outputs are affected by temperature, humidity and the age of the heating element, whereas a monitoring station uses calibrated instruments.

6.5 Limitations

- MQ sensors are not selective. MQ135 and MQ5 readings are a mixture of gases, and individual concentrations are estimates.
- The DHT11 resolves only whole degrees and has modest accuracy.
- Calibration used datasheet constants and was not checked in a gas chamber.
- The dust sensor was not calibrated against a reference device, so particulate data is excluded from AQI.
- The test run was short, and long-term drift was not studied.

7. Smart City Application Perspective

A single node gives information about one point, while a city needs many. Figure 15 of the research paper sketches copies of the node at traffic junctions, residential zones, industrial areas, and sensitive sites such as schools and hospitals, all reporting to one cloud channel group. This section is a design proposal and was not tested at that scale.

- Scalability: each node needs only Wi-Fi and power. It could later use LoRaWAN without changing the sensing function.
- Utility of data: readings from several districts can feed a city dashboard, public alerts and decisions such as routing or construction.
- Citizen participation: because the hardware is cheap, schools and residents' associations can run their own nodes.
- Accuracy: low-cost nodes should be synchronised periodically with a reference station and re-calibrated.

8. Conclusion and Future Work

An IoT air pollution sensor node was designed and built with an Arduino Uno, DHT11, MQ5, MQ135 and an ESP8266 connected to ThingSpeak. The prototype gave continuous measurements on the dashboard and a local alarm when a threshold was exceeded. Its readings agreed with reference temperature and AQI values to a good extent, which makes it suitable for awareness purposes.

Future work includes: adding a calibrated particulate matter channel; testing the sensors against a reference station; a machine learning model for pollution episode prediction; and a mobile application for citizens.

9. References

[1] World Health Organization, “Ambient (outdoor) air pollution,” WHO Fact Sheet, Geneva, Switzerland.

[2] World Health Organization, “Air pollution and child health: Prescribing clean air,” Geneva, Switzerland, 2018.

[3] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, “Internet of Things (IoT): A vision, architectural elements, and future directions,” Future Gener. Comput. Syst., vol. 29, no. 7, pp. 1645–1660, Sep. 2013.

[4] A. R. Kondeti, L. Rudraksha, S. Chinnaiahgari, and A. Bujunuru, “Real-time air quality and weather monitoring system utilizing IoT for sustainable urban development and environmental management,” Eng. Proc., vol. 118, no. 1, p. 56, Nov. 2025.

[5] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, “Internet of Things for smart cities,” IEEE Internet Things J., vol. 1, no. 1, pp. 22–32, Feb. 2014.

[6] J. J. Caubel et al., “A new black carbon sensor for dense air quality monitoring networks,” Sensors, vol. 18, no. 3, 2018.

[7] G. Lo Re et al., “Mobile application for air pollution exposure,” IEEE Sensors J., vol. 13, no. 10, pp. 3965–3974, 2013.

[8] A. Jayakumar, “IoT based air pollution monitoring system,” Int. Res. J. Eng. Technol., 2021.

[9] H. Gupta et al., “IoT based air pollution monitoring for smart cities,” in Proc. Int. Conf. Comput. Commun. Autom., 2019, pp. 1–5.

[10] M. Singh et al., “IoT based air monitoring using Arduino,” Int. Res. J. Eng. Technol., vol. 6, no. 3, pp. 1–4, 2019.

[11] N. D. Desai et al., “IoT monitoring using BeagleBone Black,” in Proc. Int. Conf. Trends Electron. Informat., 2017, pp. 1–6.

[12] K. S. E. Phala et al., “Air quality monitoring system standards,” in Proc. IEEE Int. Conf. Ind. Technol., 2016, pp. 2038–2043.

[13] P. Pal et al., “IoT pollution monitoring system,” Int. Res. J. Eng. Technol., vol. 4, no. 5, pp. 1–5, 2017.

[14] S. Dhingra et al., “IoT mobile air monitoring system,” IEEE Internet Things J., vol. 6, no. 3, pp. 5577–5584, 2019.

[15] M. Anitha and L. S. Kumar, “Development of an IoT-enabled air pollution monitoring and air purifier system,” MAPAN, vol. 38, pp. 1–12, May 2023.

[16] Central Pollution Control Board, “National Air Quality Index,” Ministry of Environment, Forest and Climate Change, Government of India, New Delhi, 2014.

[17] Hanwei Electronics, “MQ-135 gas sensor technical data sheet,” Zhengzhou, China.

[18] Hanwei Electronics, “MQ-5 gas sensor technical data sheet,” Zhengzhou, China.

[19] MathWorks, “ThingSpeak IoT analytics platform documentation,” Natick, MA, USA.